

My Code

Card.hpp

```
#include <iostream>
#include "CardTypes.h"
#pragma once
using std::cout;

class Card
{
public:
    Card(Rank rank = Ace, Suit suit = Spades)
    {
        m_rank = rank;
        m_suit = suit;
    }
    void display()
    {
        cout << m_rank << " of " << m_suit << "\n";
    }

    void setRank(Rank rank) { m_rank = rank; }
    void setSuit(Suit suit) { m_suit = suit; }

    Rank getRank() { return m_rank; }
    Suit getSuit() { return m_suit; };

private:
    Rank m_rank;
    Suit m_suit;

};
```

Deck.hpp

```
#include <algorithm>
#include <ctime>
#include <cstdlib>
#include <random>
#include "Card.hpp"
#include "CardTypes.h"
#pragma once

class Deck
{
private:
    Card m_deck[52];
    int m_currentCard;

    int getRand() { return rand(); }
public:
    Deck()
    {
        int suit = 0;
        int rank = 1;
```

```

        for (int i = 0; i < 52; i++)
        {
            m_deck[i].setRank((Rank)rank);
            m_deck[i].setSuit((Suit)suit);
            if (rank == 13)
            {
                rank = 1;
                suit++;
            }
            else {
                rank++;
            }
        }
        m_currentCard = 51;
    }

    void shuffle()
    {
        std::default_random_engine rng(static_cast<unsigned int>(std::time(0)));
        std::shuffle(m_deck, m_deck + 52, rng);
        m_currentCard = 51;
    }

    Card deal()
    {
        if (m_currentCard < 0)
        {
            cout << "No more cards left in the deck\n";
            return Card();
        }
        return m_deck[m_currentCard--];
    }
};

```

CardTypes.h

```
#pragma once
```

```

enum Rank
{
    Ace = 1,
    Deuce,
    Trey,
    Four,
    Five,
    Six,
    Seven,
    Eight,
    Nine,
    Ten,
    Jack,
    Queen,
    King
};

```

```

enum Suit
{
    Spades,
    Hearts,

```

```
    Clubs,  
    Diamonds  
};
```

AI Feedback

Here's a careful review of your Deck of Cards implementation, along with suggestions and improvements.

AI Evaluation of Your Deck of Cards Code

✓ Strengths

1. Follows UML structure closely

- Deck contains an array of Cards (m_deck[52]).
- Card contains Rank and Suit .

2. Proper card initialization

- Loops correctly to assign all 52 cards.
- Ranks reset after 13 and suits increment.

3. Shuffle is done with a proper random engine

- Uses std::shuffle with std::default_random_engine seeded by time(0) .

4. Deal function works

- Decrements m_currentCard and handles empty deck case.

5. Default constructors and setters/getters are correct

⚠ Issues / Improvement Opportunities

1. Use of rand() in getRand() is unused

- The getRand() function isn't used anywhere.
- You're already using std::shuffle with std::default_random_engine , so getRand() can be removed.

2. display() prints numeric enums

```
cout << m_rank << " of " << m_suit << "\n";
```

- Will print numbers like 1 of 0 .
- Consider mapping enums to human-readable strings:

```
const char* rankStr[] = {"", "Ace", "Deuce", ..., "King"};
```

```
const char* suitStr[] = {"Spades", "Hearts", "Clubs", "Diamonds"};
cout << rankStr[m_rank] << " of " << suitStr[m_suit] << "\n";
```

3. `m_currentCard` indexing may be confusing

- You initialize `m_currentCard = 51`, then decrement.
- Alternative: start at 0 and increment; clearer in some contexts.

4. `card` getters should be `const`

```
Rank getRank() const { return m_rank; }
Suit getSuit() const { return m_suit; }
```

5. Use `enum class` instead of plain enums

- Prevents accidental implicit conversion to integers.
- Example:

```
enum class Rank { Ace=1, Deuce, ..., King };
enum class Suit { Spades, Hearts, Clubs, Diamonds };
```

6. `Deck` could have a `reset()` function

- To reinitialize a full deck without creating a new `Deck` object.

7. `display()` could be part of `Deck`

- Print the whole deck in order for debugging.

Improved Version Snippet

Here's a cleaner version with readable `display()` and minor adjustments:

```
#include <algorithm>
#include <ctime>
#include <random>
#include <iostream>

enum class Rank { Ace=1, Deuce, Trey, Four, Five, Six, Seven, Eight, Nine, Ten, Jack, Queen, King };
enum class Suit { Spades, Hearts, Clubs, Diamonds };

class Card {
public:
    Card(Rank rank = Rank::Ace, Suit suit = Suit::Spades)
        : m_rank(rank), m_suit(suit) {}

    void display() const {
        const char* rankStr[] = { "", "Ace", "Deuce", "Trey", "Four", "Five", "Six", "Seven",
                                   "Eight", "Nine", "Ten", "Jack", "Queen", "King" };
        const char* suitStr[] = { "Spades", "Hearts", "Clubs", "Diamonds" };
        std::cout << rankStr[static_cast<int>(m_rank)] << " of "
```

```

        << suitStr[static_cast<int>(m_suit)] << "\n";
    }

    void setRank(Rank rank) { m_rank = rank; }
    void setSuit(Suit suit) { m_suit = suit; }
    Rank getRank() const { return m_rank; }
    Suit getSuit() const { return m_suit; }

private:
    Rank m_rank;
    Suit m_suit;
};

class Deck {
public:
    Deck() { initialize(); }

    void shuffle() {
        std::default_random_engine rng(static_cast<unsigned int>(std::time(nullptr)));
        std::shuffle(m_deck, m_deck + 52, rng);
        m_currentCard = 51;
    }

    Card deal() {
        if (m_currentCard < 0) {
            std::cout << "No more cards left in the deck\n";
            return Card();
        }
        return m_deck[m_currentCard--];
    }

    void displayDeck() const {
        for (int i = 0; i <= m_currentCard; i++) {
            m_deck[i].display();
        }
    }

private:
    Card m_deck[52];
    int m_currentCard;

    void initialize() {
        int rank = 1;
        int suit = 0;
        for (int i = 0; i < 52; i++) {
            m_deck[i].setRank(static_cast<Rank>(rank));
            m_deck[i].setSuit(static_cast<Suit>(suit));
            if (rank == 13) {
                rank = 1;
                suit++;
            } else {
                rank++;
            }
        }
        m_currentCard = 51;
    }
};

```

✓ This version:

- Prints cards as "Ace of Spades" .
- Uses `const` correctness.
- Removes unused `getRand()` .
- Includes a `displayDeck()` function.
- Ready to integrate into any card game.